

Side-Channel Leakage in Low-Entropy Masking Schemes Attributed to FPGA Architecture

Yanbin Li¹, Yikang Guo, and Fan Zhang², *Member, IEEE*

Abstract—Masking is one of the most popular to guarantee the security of the cryptographic implementation against side-channel analysis (SCA). Low-entropy masking scheme (LEMS) has been proposed to relieve the high overhead by reducing the entropy of the mask. The masks are selected carefully to resist univariate first-order attacks. However, in practice, securely implementing masking can be a challenging task. In this work, we exhibit the vulnerability of LEMS implementation on field programmable gate array (FPGA). First, we provide the security model to characterize the masking balance at the implementation level. Based on the security model, we exhibit a first-order leakage in the netlist of the implementation. It can be proved that this defect is due to the uncontrollable electronic design automation (EDA) tool to optimize the advanced encryption standard (AES) Sbox implementation on the specific architecture of FPGA during synthesis. The discovered flaw can be exhibited by performing a couple of first-order attacks to recover the secret key successfully on FPGA. The vulnerability is verified by simulation and practical measurements. Finally, the potential countermeasure and security evaluation process for LEMS are provided to avoid leakage during the pre-silicon design phase.

Index Terms—Advanced encryption standard (AES), field programmable gate array (FPGA), low-entropy masking scheme (LEMS), side-channel analysis (SCA).

I. INTRODUCTION

SIDE-CHANNEL analysis (SCA) has emerged as a serious threat to the cryptographic algorithms by exploiting the physical leakage. It includes the running time, power consumption, or electromagnetic radiation during the cryptographic algorithm execution. Due to the threat to ASIC, and field programmable gate array (FPGA) implementation of the cryptographic algorithm, many different countermeasures have been considered and developed.

Received 1 November 2024; revised 15 April 2025 and 20 July 2025; accepted 2 September 2025. Date of publication 8 September 2025; date of current version 23 March 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106800; in part by the National Natural Science Foundation of China under Grant 62472259, Grant 62572285, Grant 62072247, Grant 62032025, and Grant 62172258; in part by the Program of Taisihan Young Scholars of Shandong Province under Grant tsqn202408026; and in part by Shandong Provincial Natural Science Foundation under Project ZR2024QF098. This article was recommended by Associate Editor E. B. Kavun. (Corresponding author: Fan Zhang.)

Yanbin Li is with the School of Software, Shandong University, Jinan 250100, China, and also with the State Key Laboratory of Cryptology, Beijing 100878, China.

Yikang Guo is with the College of Artificial Intelligence, Nanjing Agricultural University, Nanjing 210095, China.

Fan Zhang is with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310007, China, and also with Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Hangzhou 311121, China (e-mail: fanzhang@zju.edu.cn).

Digital Object Identifier 10.1109/TCAD.2025.3607151

Masking is a well-known effective countermeasure to protect the implementations of cryptographic algorithms against SCAs. The idea of masking is to introduce random numbers to make the observable physical leakage independent of sensitive information. A provable security model was proposed by Ishai et al. [1], which assumed that any d intermediate variables could be probed in the masking scheme. Later, there were many higher order masking schemes proposed to achieve d th-order security [2], [3], [4], [5], [6]. Along with the higher security, the overhead of a higher order masking scheme is nonnegligible and implementing it in the constrained device is a challenging task.

To seek the trade-off between security and overhead, reducing the entropy of the mask has been utilized to relieve the high overhead. This type of masking scheme is called the low-entropy masking scheme (LEMS) [7], [8], [9]. Compared to generating masks randomly from $\text{GF}(2^n)$ in full-entropy masking schemes (FEMS), the masks are selected from the subset of $\text{GF}(2^n)$ in LEMS. The rotating Sbox masking (RSM) was proposed based on the look-up table (LUT)-based [7], aiming at resistance of first-order differential power analysis attack (DPA)/correlation power analysis attack (CPA) with low overhead. RSM was also adopted as the countermeasure of advanced encryption standard (AES) in DPA contest v4 [9].

The security of RSM and its variants [9], [10], [11], [12], [13] has been investigated for a long time. In the implementation of RSM, the input mask of Sbox m_i and the output mask m_{i+1} share the same register, which makes the power consumption related to $m_i \oplus m_{i+1}$. This unbalanced feature was exploited by Moradi et al. to perform the univariate correlation power attack [14]. Ye and Eisenbarth [10] proposed the leakage distribution decomposition attack and the leaking set collision attack to retrieve the secret key in LEMS. Grosso et al. [15] illustrated that the higher degree terms in the leakage function influenced the security order of LEMS. Besides, performing the first-order attack combining multiple bits recovered the secret key successfully [13]. The importance of a proper mask set in LEMS was analyzed and summarized in [9].

While research on LEMS mainly focused on the balance of mask set and leakage distribution, few efforts analyzed the security of the masking implementation on FPGA. Moradi and Immler [16] analyzed the security of DPA-resistant logic styles on FPGA and provided solutions for imbalanced routing and early propagation. The suitability against first-order attacks by randomized LUTs was evaluated under different Xilinx FPGA memory primitives [17]. So far, there is

no work on the security of LEMS on the specific FPGA architecture.

The AES Sbox can be considered an 8-bit input and 8-bit output Boolean function. In RSM, the masked Sbox is implemented as a LUT with 8 inputs and 8 outputs, which is transformed from the original Sbox according to the mask value. However, the maximum support is 6-input LUT in modern FPGAs, including Xilinx Virtex-5, -6, -7, and Spartan-6. Regarding LUT-based FPGA implementations, the constructions of the masked Sbox should be optimized to map the available slices on FPGA. The security of LEMS during the optimization by uncontrollable FPGA tools has not been discussed.

This work belongs to solving the security gap between schemes and implementations. Securely implementing masking can be a challenging task. Several works have shown that the glitch could make the masking scheme fail to achieve its security order in the masked circuits [18], [19], [20]. The masked gates were vulnerable to first-order attacks because of the glitches [18]. The toggle-count model was used as the leakage model to recover the secret key targeting the masked Sbox [19], [20]. Moreover, another type of leakage occurs during synthesis optimization by standard electronic design automation (EDA) tools [21]. As shown in [21], the implementation of SecMult algorithm had a pitfall after synthesis [2]. The masking circuit was implemented by 4-input LUTs that leaked the sensitive information in the output of 4-input LUT. The security flaw found in this work is also related to the EDA optimization on the FPGA architecture.

In this work, we present the vulnerability of LUT-based LEMS on FPGA for the first time, which is influenced by the inherent specific architecture. The security is investigated from the perspective of the balance of the mask set. Even if the mask set is balanced in a masking scheme, it may not achieve the security order at the implementation level. The main contributions of this work include.

- 1) The bit-balance (BB) model was proposed in [22] to characterize the balance of the mask set. We extend it to the hardware implementation, named *BB implementation (BBI)* model, to evaluate the security at the implementation level.
- 2) The vulnerability of the FPGA implementation of RSM is exhibited under *BBI* model. We analyze the discovered flaw that occurred by the uncontrollable EDA tool to optimize the AES Sbox implementation on the specific architecture of FPGA during synthesis.
- 3) Experiments with simulation and practical measurements from the FPGA implementation of RSM are conducted to verify the security flaw. Moreover, the potential countermeasure and security evaluation process for LEMS are provided to avoid leakage during the pre-silicon design phase.

The remainder of this article is organized as follows. Section II demonstrates the definitions in LEMS and describes the RSM scheme. Besides, we show the power consumption model of FPGA, which is used in the following security analysis. Section III provides *BBI* model and shows the potential

vulnerability in LEMS on FPGA. The simulation and practical attacks on FPGA are provided in Section IV. The discussion of possible countermeasures and security evaluation process is provided in Section V. Section VI concludes the article.

II. PRELIMINARIES

A. Low-Entropy Masking Schemes

To reduce the high cost of masking to resist side-channel attacks, LEMSs are designed to maintain the security of the cryptographic implementation. Like other masking schemes, it draws masks from a subset of $\text{GF}(2^n)$ to randomize the intermediate value, where n denotes the bit-length of the intermediate variable in a cryptographic algorithm.

Compared with the FEMS, protecting the sensitive information with the masks from the entire set, LEMS saves lots of repeated computation for the nonlinear operation. For a well-designed LEMS implementation, its performance and computational complexity are acceptable to the practical application. The definitions of the full entropy mask set and low-entropy mask set are also given in [13].

Definition 1 (Full entropy mask set): Let $\mathbb{M}$ be an n -bit mask set and M be the mask variable. $H(M)$ denotes the entropy of M , where $M \in \mathbb{M}$. $\mathbb{M}$ is a full entropy mask set if $H(M) = n$.

Definition 2 (Low-entropy mask set): Let $\mathbb{M}$ be an n -bit mask set and M be the mask variable. $H(M)$ denotes the entropy of M , where $M \in \mathbb{M}$. $\mathbb{M}$ is a low-entropy mask set if $H(M) \in (0, n)$.

Generally, the complexity of masking operations in LEMS is lower because it needs less randomization than FEMS. Resisting side-channel attacks such as CPA and DPA requires carefully designing the scheme. Since the masks in the mask set are selected from a subset, one critical condition to achieve first-order security is that each bit of the masked variables should be independent of the unmasked bits.

B. RSM Scheme

RSM and its variants [7], [8], [9] are typical of the LEMS implementations. It can resist first-order SCA attacks, second-order SCA attacks, and other attacks at the cost of consuming a small amount of area and performance by limiting the value range of the random mask to 16 fixed elements. In order to guarantee security, it is necessary to achieve balance when choosing a random mask in the fixed set. Each bit of the mask state has the same probability of being equal to 0 and 1.

For the implementation of AES-128, it needs $l = 10$ rounds of encryption. It selects 16 values in the mask set, noted as

$$\mathbb{M} = \{m_0, m_1, m_2, m_3, m_4, m_5, m_6, m_7, m_8, m_9, m_{10}, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}\}. \quad (1)$$

RSM rotates Sboxes with these masks for the entire computation of the AES algorithm. The 16 masked Sboxes are precomputed by

$$\text{MSbox}_i(X) = S(X \oplus M_i) \oplus M_{i+1}. \quad (2)$$

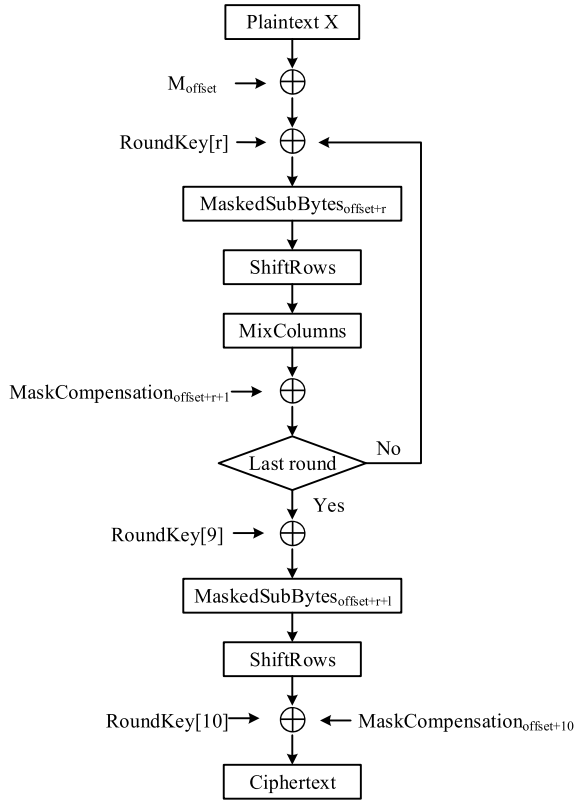


Fig. 1. RSM datapath [7].

The masks are determined by the random offset drawing from $[0, 15]$ at the beginning of the computation. The datapath of RSM implementation is shown in Fig. 1. It uses MaskCompensation to unmask the data and remask it to ensure correctness simultaneously. The implementation performs the masking transformations on other operations, such as ShiftRows and MixColumns. For these linear properties, it only requires a simple XOR operation. Finally, it can obtain the ciphertext utilizing the masking scheme.

C. Power Consumption Model of FPGA

FPGA logic is built on the CMOS transistor pairs at the circuit level. Thus, the power consumption of an FPGA satisfies the power characterize of a CMOS circuit.

The power consumption of CMOS includes static power and dynamic power. The switching power plays an important role in dynamic power of the circuit. The main current power simulation tools integrate the power calculation of the design, such as PrimePower and PrimeTime PX. The switching power P_c can be calculated [23] for every rising transition and divided by the total time

$$P_c = \frac{V_{dd}^2}{2} \cdot \sum_i (C_{Load_i} \cdot TR_i) \quad (3)$$

where V_{dd} represents the supply voltage, and C_{load_i} and TR_i represent the capacitive load and toggle rate of net i , respectively.

In the side-channel area, it had been simplified as the total number of signal toggles in a circuit, named the toggle-count

model. It has been exploited to attack the unmasked and masked AES Sbox successfully [20], [24], [25]. Based on (3), the toggle-count model represents the crucial factor of data-dependent power, which is used as the power consumption model of FPGA in this work.

III. SECURITY FLAW OF LEMS ON FPGA

In this section, we first summarize the related attacks on the mask sets of RSM. Then, recall the *BB Security* model [22] and extend it to the hardware implementation. Then, we show the FPGA architecture and provide the AES Sbox implementation of FPGA. Finally, the leakage in the FPGA implementation of RSM based on the security model is presented.

A. Security Model of LEMS

The balance of the mask set is the essential condition to resist the first-order CPA. The mask set is continuously updated and improved as research progresses

MaskSet₁

$$= \{0x00, 0x0f, 0x36, 0x39, 0x53, 0x5c, 0x65, 0x6a, 0x95, 0x9a, 0xa3, 0xac, 0xc6, 0xc9, 0xf0, 0xff\}$$

MaskSet₂

$$= \{0x03, 0x0c, 0x35, 0x3a, 0x50, 0x5f, 0x66, 0x69, 0x96, 0x99, 0xa0, 0xaf, 0xc5, 0xca, 0xf3, 0xff\}$$

MaskSet₃

$$= \{0x13, 0x94, 0x25, 0xcb, 0x8e, 0x5f, 0xd9, 0x37, 0x77, 0xc6, 0xa8, 0x38, 0x05, 0xea, 0x70, 0xe8\}$$

MaskSet₄

$$= \{0x00, 0x0f, 0x36, 0x39, 0x53, 0x95, 0x5c, 0xc9, 0xff, 0xc6, 0xac, 0x9a, 0x6a, 0xa3, 0x65, 0xf0\}. \quad (4)$$

Currently, there are four mask sets for RSM schemes. The first mask set MaskSet₁ was proposed in DPA contest v4.1 [9], however, the subsequent works showed that it cannot resist the first-order CPA attack for the vulnerabilities in the mask set [14], [26]. The MaskSet₂ was used in DPA contest v4.2, which was attacked by exploiting a bivariate vulnerability [11]. The MaskSet₃ proposed by Veshchikov and Guilley [26] and MaskSet₄ proposed by Moradi et al. [14] had not been attacked, until Ming et al. [13] showed the attacks exploiting the multivariable leakage. It exhibited that there is no secure mask set for RSM to achieve the expected security level under multivariable leakage assumption.

In this work, we do not focus on the bivariate, multivariable leakage assumption, or second-order attacks. This work primarily reveals the leakage caused by the FPGA architecture, which can be demonstrated even under the simplest univariate leakage model, that is, single-bit model or Hamming weight (HW) model. This section takes the MaskSet₄ as example.

The *BB security* model was defined to evaluate the first-order BB security of the mask set in LEMS [22]. It also considered the univariate leakage model. The MaskSet₄ is validated and considered to resist the first-order DPA/CPA.

Definition 3 (BB Security [22]): Assume M is the random variable of mask in $\mathbb{M}$, X is the n -bit unmasked variable and X_M represents the masked variables after masking operation. The unmasked variable X is uniformly distributed over $\text{GF}(2^n)$. $X[k]$ and $X_M[k]$ denote the k th-bit of its variable, where $k \in [0, n-1]$. $\forall k \in [0, n-1]$, if $E(X_M[k]|X[k] = 0) = E(X_M[k]|X[k] = 1)$, the masking scheme with mask set $\mathbb{M}$ is BB secure.

The *BB security* model reflects the balance of masked variables at the algorithm level. In the actual attack, there is always a gap between the algorithm and practical implementation. We extend the *BB security* model to the hardware implementation scene.

The implementation of LEMS is designed and considered to resist first-order DPA/CPA. Assume an implementation of the cryptographic algorithm, Q is the input or output and S represents the secret key. The sensitive information is $X = f(Q, S)$, where f denotes the crucial function in a cryptographic algorithm. In the masked implementation, the mask M is generated from a set $\mathbb{M} = \{m_0, m_1, \dots, m_{|\mathbb{M}|}\}$ so that the observable leakage L only relates to the masked information $X_M = \{f_M(Q, S)\}$, where the masked function f_M should be designed and implemented carefully. The power consumption L can be represented as

$$L = H(X_M) + \varepsilon = H(f_M(Q, S)) + \varepsilon \quad (5)$$

where $H(\cdot)$ represents the leakage model and ε denotes a gaussian noise.

The observable leakage L from the implementation should be independent of X . The statistic dependency between L and X can be described as

$$L \perp X \implies \text{Var}[E(L|X)] = 0 \quad (6)$$

where $\perp$ represents the statistical independence, E represents the expectation, and Var represents the variance. The expectation is computed by the mean of L for each class x taken by X . The variance measures the interclass variance of L in each class $X = x$. The CPA calculates the Pearson correlation coefficient between the sensitive information and observable leakage. When $\rho(L, X) = \rho(E(L|X), X) \neq 0$, the $E(L|X = x)$ does not vary with the value x . That means the interclass variance of the expectation is zero, which can resist the CPA attack. For a countermeasure, the balance of masking implementation should be satisfied to avoid interclass variance, that is, $\text{Var}[E(L|X)] = 0$ [27]. In the following, the *BB security* model is extended to the hardware implementation of LEMS.

Definition 4 (BB Implementation Security): Assuming f is the function that outputs the masked variable X_M , circuit C_f implements the function. The unmasked variable X is uniformly distributed over $\text{GF}(2^n)$. $X[k]$ and $X_M[k]$ denote the i th-bit of its variable, where $i \in [0, n-1]$. $\forall k \in [0, n-1]$. Let H be the leakage model of the circuit C_f . If $E(H(X_M[k])|X[k] = 0) = E(H(X_M[k])|X[k] = 1)$, the masking scheme with mask set $\mathbb{M}$ is BBI secure.

BBI model implies the implementation security in the perspective of bit balance to resist first-order CPA/DPA.

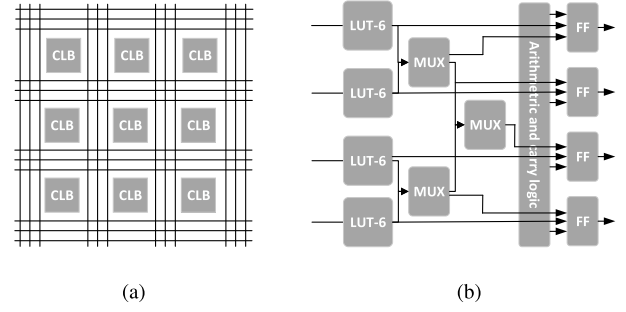


Fig. 2. CLB arrangement and slice description. (a) General block diagram of FPGA. (b) Diagram of slice.

B. FPGA Architecture

FPGA is the reconfigurable hardware consisting of configurable logic blocks named CLB. The CLB is the basic building block containing logic cells, shown in Fig. 2(a). The CLBs are connected through programmable interconnects of wire segments. They are organized into rows and columns and assigned coordinate values.

The CLB of modern Xilinx FPGAs contains two slices. In Fig. 2(b), it can be found that the slice includes four LUTs, carry logic, and registers. The LUT is the basic building block that can be used to implement logic functions. It implements different functions to output values by the truth table with different combinations of the inputs.

For a general N -input LUT, the number of addresses accessed by the truth table is 2^N , where N represents the number of inputs. The traditional Spartan-3 and Virtex-4 FPGAs maximum support 4-input LUTs, while modern FPGAs support 6-input LUTs, for example Spartan-6, Virtex-5, -6, and -7.

AES is a well-known symmetric cryptographic algorithm. By default, it refers to AES-128 in this article. It requires ten rounds, including Sbox, ShiftRows, MixColumns, and AddRoundKey. The Sbox is the most consuming operation. The input size and output of Sbox are 8 bits. Its LUT implementation needs $2^8 = 256$ input addresses to map the correct output value. However, the mainly modern FPGAs maximum support 6-input LUTs. Instead, its Sbox implementation of FPGA uses the 6-input LUTs combined with multiplexers [28]. The implementation structure of Virtex-5 can be seen in Fig. 3.

Let $x = x_7x_6x_5x_4x_3x_2x_1x_0$ represents the input of Sbox operation and $y = y_7y_6y_5y_4y_3y_2y_1y_0$ represents the output. We take the implementation of output y_0 as an example. Fig. 3 shows the implementation of calculation the output bit y_0 . Theoretically, one byte Sbox can be implemented by eight pieces of the circuit. The structure for 8 bits is the same, while the truth tables in the LUTs are different. It needs $2^8 = 256$ addresses in a LUT, replaced by four 6-input LUTs. Moreover, the bit x_6 and x_7 are used as selection signals in multiplexers to indicate the correct output.

Instead of a simple implementation of one 8-input LUT, it is implemented by a more complex circuit including four LUTs and three multiplexers. The power consumption of this circuit is related to the activities of the components. It only

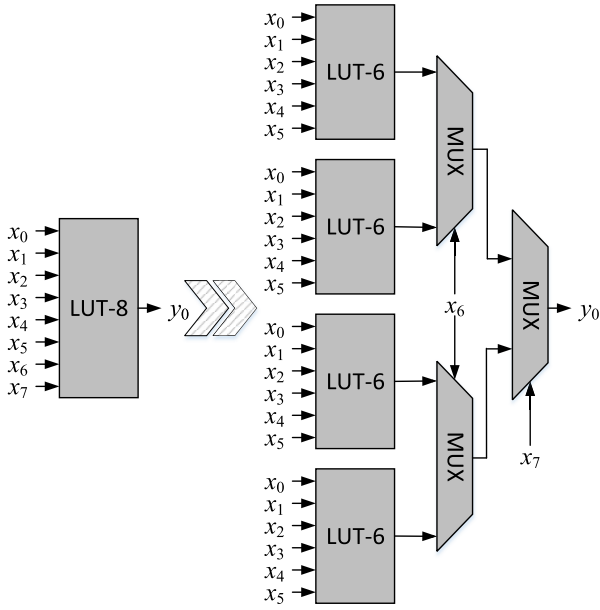


Fig. 3. Implementation of AES Sbox on FPGA. $y_7y_6y_5y_4y_3y_2y_1y_0 = \text{Sbox}(x_7x_6x_5x_4x_3x_2x_1x_0)$. This figure represents the output y_0 . The structure for other bits are the same with y_0 .

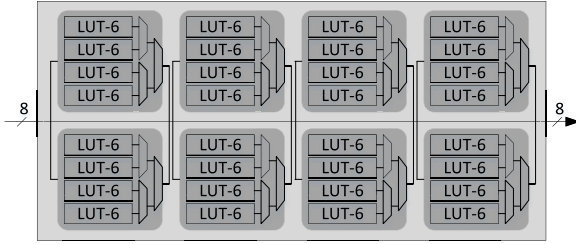


Fig. 4. Implementation architecture of the masked Sbox of RSM.

restricts the output signal y_0 to be balanced under *BB* model. Obviously, the characteristics of the whole circuit need to be considered in the actual implementation of FPGA.

C. Leakage Analysis

RSM carefully chooses the mask values to achieve *BB security*. However, this security stays at the algorithm level. The power character is related to the underlying implementation.

The RSM scheme is implemented on Xilinx Spartan-6 by ISE 14.4. The implementation of the masked Sbox is shown in Fig. 4. The 16 masked Sboxes are precomputed according to (2). The masked Sbox is implemented in the combinational logic within one clock cycle. Each of them is an 8-to-8-bit function, which can be considered eight 8-to-1-bit functions. During the synthesis of EDA tool, the inputs of LUTs for the masked Sbox are replaced by the bits of the masked variable and the truth tables are initialed corresponding to the masked Sbox automatically. Each 8-to-1-bit function is synthesized into four 6-input LUTs occupying one slice. The 8-to-1-bit LUT of masked Sbox requires 32 LUTs, all fitting into eight slices.

To analyze the leakage of the implementation, it needs to access the detailed description of the circuit. The implementation details can be found in the netlist at the place and route

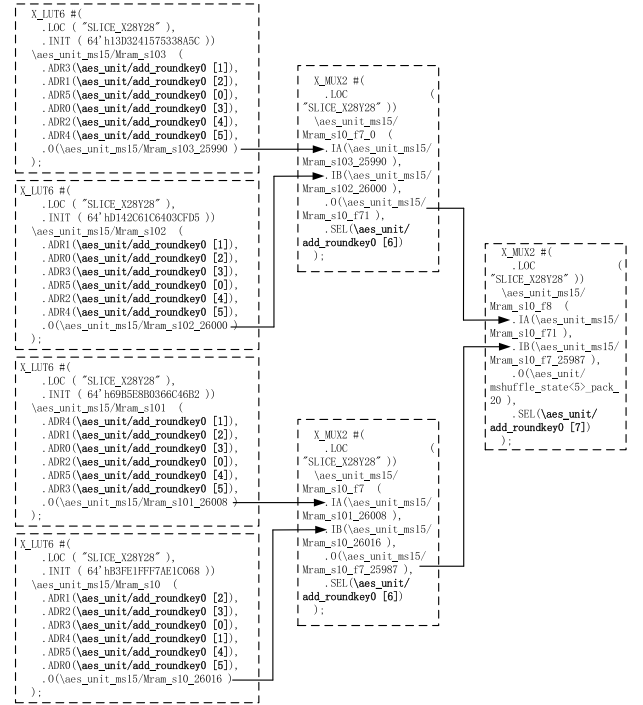


Fig. 5. First mode of implementation structure of the masked Sboxes.

level. There are 16 masked Sboxes during one round total. The 16 masked Sboxes are synthesized into two structure modes.

The first mode of implementation structure is shown in Fig. 5. The `\aes_unit/add_roundkey0` is the input of masked Sbox, representing the masked state after the plaintext XOR key. In the implementation of 1st-byte masked Sbox, the six bits `\aes_unit/add_roundkey0[5:0]` are used as the inputs of 6-input LUTs, while `\aes_unit/add_roundkey0[6]` and `\aes_unit/add_roundkey0[7]` are the select signals for the multiplexors. Through analyzing the netlist, it can be found that there are 14 bytes masked Sboxes implemented in the first mode, that is, the low six bits are used as the inputs of LUTs and the high two bits are the select signals for the multiplexors.

Another mode is the irregular implementation of 2nd-byte and 13th-byte masked Sboxes. The implementation of the 2nd-byte masked Sbox is shown in Fig. 6. The high six bits `\aes_unit/add_roundkey0[15:10]` of the masked state are used as the inputs of 6-input LUTs, while the two remaining bits `\aes_unit/add_roundkey0[8]` and `\aes_unit/add_roundkey0[9]` are the select signals for the multiplexors. This difference in implementations may occur by uncontrollable EDA tool, which aims at design optimization. It can be found that the 13th-byte masked Sbox takes the `\aes_unit/add_roundkey0[98]` and `\aes_unit/add_roundkey0[99]` as the select signals while the remaining six bits as the input of LUTs.

There are 16 masked Sboxes in the RSM scheme. The masked input of masked Sbox $x[0 : 7]$ is randomized by a uniformly distributed random number. The first mode uses low bits $x[0 : 5]$ as the input of LUT-6 and high bits $x[6 : 7]$ as the select signals for the multiplexors. On the contrary, the second

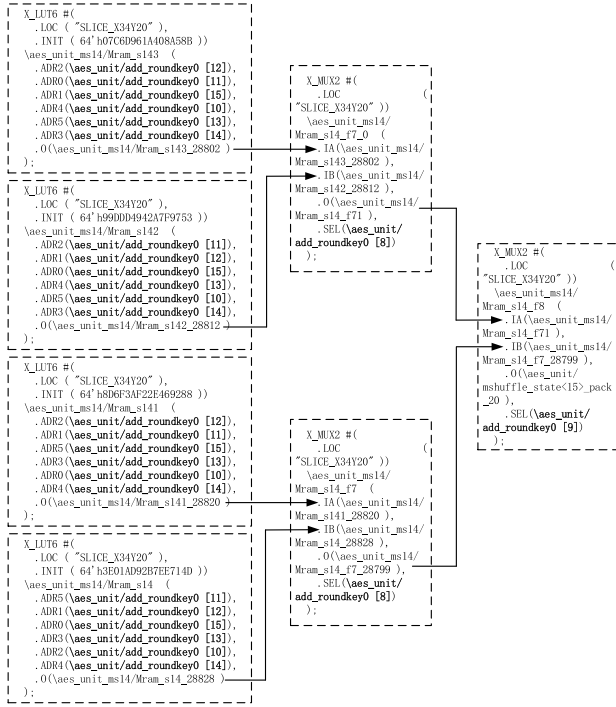


Fig. 6. Second mode of implementation structure of the masked Sboxes.

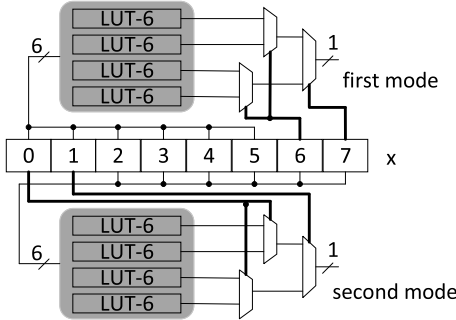


Fig. 7. Inconsistent structure of the masked Sbox.

mode uses high bits $x[2:7]$ as the input of LUT-6 and high bits $x[0:1]$ as the select signals for the multiplexors, shown in Fig. 7. The bold lines indicate the select signals. Similarly, 13th-byte masked Sbox takes $x[2:3]$ as the select signals and the other bits as the input of LUT-6. The inconsistent structure causes differences in internal states.

The initial truth table of LUTs can be designed by the inputs to guarantee the correct output of the masked Sbox. Thus, those different implementations cannot destroy the correctness. However, the composition of the circuit under different masks has changed, which will also influence the power consumption of the masked circuits. Proposition 1 is proposed to discuss how it affects the security of LEMS.

Proposition 1: Assume circuit C_f implements the output of masked Sbox bit $X_M[k]$ by taking X_{in} as the input signals, $k \in [0, n-1]$. The unmasked variable X is uniformly distributed over $GF(2^n)$. In the circuit of i th-masked Sbox, the bits X_{LUT}^i are used as the input signals of 6-input LUT and $X_{MUX}^i = \{X_{MUX1}^i, X_{MUX2}^i\}$ are used as the select signals for the two multiplexors, that is, $X_{in}^i = X_{LUT}^i \cup X_{MUX}^i$. If there exists

different implementation structures of 16 masked Sbox, that is, $\exists i, j, X_{LUT}^i \neq X_{LUT}^j, X_{MUX}^i \neq X_{MUX}^j$, the BB secure masking scheme cannot achieve BBI security.

Proof: Based on the toggle-count model, the power consumption $H(X_M[k])$ is related to the transitions in the circuit

$$H(X_M[k]) = \sum_{r=0}^3 t_{LUT}^r + \sum_{r=0}^2 t_{MUX}^r$$

where t_{LUT}^r represents the toggle of r th-LUT and t_{MUX}^r represents the toggle of r th-multiplexor.

The output toggle of four 6-input LUT ($LUT_0, LUT_1, LUT_2, LUT_3$) can be represented as

$$\begin{aligned} t_{LUT}^0 &= LUT_0(X_{LUT}) = \text{MSbox}(00|X_{LUT}) \\ t_{LUT}^1 &= LUT_1(X_{LUT}) = \text{MSbox}(01|X_{LUT}) \\ t_{LUT}^2 &= LUT_2(X_{LUT}) = \text{MSbox}(10|X_{LUT}) \\ t_{LUT}^3 &= LUT_3(X_{LUT}) = \text{MSbox}(11|X_{LUT}) \end{aligned}$$

where $|$ denotes the bit concatenation and $00|X_{LUT}$ is the input 8 bits of the masked Sbox. The output toggle of the multiplexor can be represented as

$$\begin{aligned} t_{MUX}^0 &= \text{If_Then_Else}(X_{MUX1} = 0, LUT_0(X_{LUT}) \\ &\quad LUT_1(X_{LUT})) \\ t_{MUX}^1 &= \text{If_Then_Else}(X_{MUX1} = 0, LUT_2(X_{LUT}) \\ &\quad LUT_3(X_{LUT})) \\ t_{MUX}^2 &= \text{If_Then_Else}(X_{MUX2} = 0, t_{MUX}^0, t_{MUX}^1). \end{aligned}$$

If_Then_Else() function returns the former value if the condition is TRUE, or the latter value if the condition is FALSE.

$$\text{If_Then_Else}(a, b, c) = \begin{cases} b, & \text{if } a = \text{TRUE} \\ c, & \text{if } a = \text{FALSE}. \end{cases}$$

Consider the situation corresponding to the inconsistency structures of masked Sbox implementations. $\exists i, j, X_{LUT}^i \neq X_{LUT}^j, X_{MUX}^i \neq X_{MUX}^j$, it has

$$\begin{aligned} E(t_{LUT}^0|X[k] = 0) &\neq E(t_{LUT}^0|X[k] = 1) \\ E(t_{LUT}^1|X[k] = 0) &\neq E(t_{LUT}^1|X[k] = 1) \\ E(t_{LUT}^2|X[k] = 0) &\neq E(t_{LUT}^2|X[k] = 1) \\ E(t_{LUT}^3|X[k] = 0) &\neq E(t_{LUT}^3|X[k] = 1). \end{aligned}$$

Thus,

$$E(H(X_M[k])|X[k] = 0) \neq E(H(X_M[k])|X[k] = 1).$$

In the above FPGA implementation of LEMS, the structures of the 2nd-byte and 13th-byte masked Sboxes differ from others. Based on proposition 1, it cannot achieve BBI security. The difference of conditional expectation (DCE) can be used to validate the bias

$$\text{DCE} = E(H(X_M[k])|X[k] = 1) - E(H(X_M[k])|X[k] = 0). \quad (7)$$

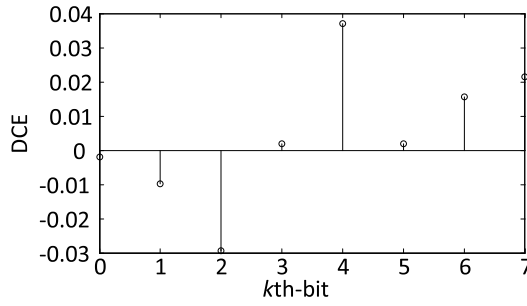


Fig. 8. DCE for each bit.

The difference result should be equal to 0 for a masking implementation against univariate first-order attacks. For each bit $k \in [0, 7]$, the DCE can be obtained based on the given implementation. The DCE for each bit is shown in Fig. 8. It exhibits the leakage of FPGA implementation of LEMS under *BBI* model. The results are consistent with Proposition 1.

IV. ATTACKS EXPLOITING THE FPGA LEAKAGE

A. Simulation Analysis

To verify the vulnerability, the side-channel attack experiments are performed on the FPGA implementation of RSM by exploiting the leakage. The experiments use Xilinx ISE design suite 14.4 to design the hardware implementation of RSM. The experiments are conducted on a 1.8 GHz Intel CPU computer.

The regular steps of ISE design include design entry, synthesis, implementation, verification, and download. The hardware description language (HDL) of RSM scheme and TESTBENCH file are used to perform the simulation. During the design process, the readable netlist file NETLIST.V is generated, which contains detailed information about the circuit. First, we analyze the implementation structure of the masked Sbox in the netlist. It can be found there are 14 bytes masked Sboxes are implemented in the way of low six bits are used as the input of LUTs and the high two bits are the select signals for the multiplexors (shown in Fig. 5), while the remaining 2 bytes masked Sboxes are not (shown in Fig. 6). According to Proposition 1, the inconsistent structure would lead to leakage of low-entropy masking on FPGA.

Second, the leakage by performing the power analysis attacks is verified on the simulation results. We run the simulation of RSM design by ISE simulator, which is a supported logic simulator in Xilinx ISE platform. The waveform data of signals is dumped to value change dump (VCD) file. In the TESTBENCH, there are 5000 ($>256 \times 16$) random plaintexts to be encrypted with a fixed secret key. The number of traces covers the space of random plaintext and masks. After performing posttranslate simulation, 5000 simulation results are generated in the VCD file. The toggle-count model is utilized to calculate the simulation power consumption of the circuit.

The sensitive value is calculated according to the original Sbox. The first-order CPA attack is performed on the FPGA implementation. In this group of experiments, we perform the attacks with HW model and single-bit model separately.

Remarkably, the implementation of RSM should be able to resist the first-order CPA or DPA theoretically [8]. However, the correct key is retrieved by the first-order CPA. That means the FPGA implementation of RSM cannot achieve first-order security under *BBI* model. It can be found in Fig. 9(a) that the black line indicates the correct key, while the gray lines indicate the wrong keys. The correct key is distinguished under HW model after 2000 traces.

In addition, we perform the CPA with the single-bit model. The difference of correlation coefficients among guessing keys is shown in Fig. 9(b)–(i). It can also retrieve the correct key under some single bits. The significant leakage can be observed in the results of bit 1, 2, 4. The difference among different bits may occur by the various delays and the noise's influence. Adopting an orthogonal array mask set is necessary to balance the masked variables in LEMS. The orthogonal array mask set is adopted in this FPGA implementation. However, the bias structure implementation makes it fail to satisfy *BBI* security.

B. Practical Analysis

The target implementation of RSM is described in Verilog and compiled with Xilinx ISE for a Spartan-6 FPGA. We download it onto the SAKURA-G board [29]. The plaintexts and keys are sent from the computer to the target board through a USB port.

The oscilloscope DSOX3024T is employed to capture and transmit the power traces. Fig. 10 depicts the power measurement setup to the FPGA core. The voltage of the main FPGA can be monitored through the SMA connector. The power traces are pretreatment by PA303 amplifier. We capture the power traces at a sampling rate of 1.25 GSa/s and a bandwidth limit of 20 MHz while the board runs at a clock frequency of 3 MHz.

The leakage assessment methodology [30], [31] is exploited to examine whether the RSM's FPGA implementation leaks sensitive information. This method can evaluate the security without attacks relaying on Welch's *t*-test. The most general form of the specific *t*-test evaluates the power traces of random plaintexts with a fixed key. The captured measurement set is split into two groups according to the intermediate value and leakage model. It examines a null hypothesis that those two groups of data are significantly different from each other.

In this practical experiment, the measured traces are partitioned into two groups L_0 , L_1 according to the sensitive variable, for example, the output of Sbox. The number of traces in the two sets is denoted as $|L_0|$ and $|L_1|$

$$t = \frac{\mu(L_0) - \mu(L_1)}{\sqrt{\frac{\sigma^2(L_0)}{|L_0|} + \frac{\sigma^2(L_1)}{|L_1|}}} \quad (8)$$

where μ represents the expectation and σ^2 represents the sample variance among the set. The statistic t follows Student's *t* distribution under the null hypothesis. For the sake of simplicity, the threshold $|t| > 4.5$ for t is often used to reject the null hypothesis, which corresponds to significance level $\alpha \leq 10^{-5}$. That means the two partition groups related

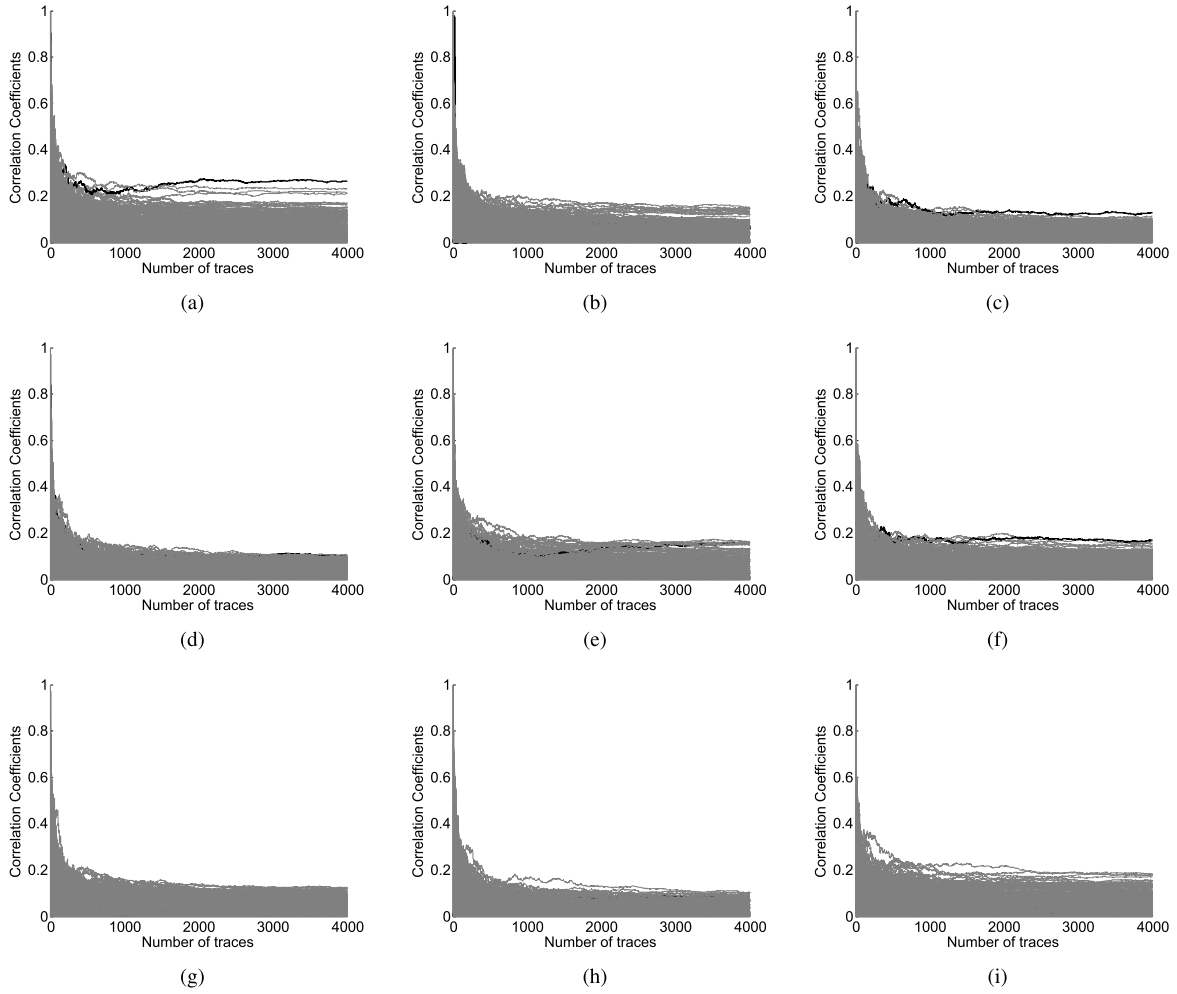


Fig. 9. Attacking results of CPA (a) HW. (b) Bit 0. (c) Bit 1. (d) Bit 2. (e) Bit 3. (f) Bit 4. (g) Bit 5. (h) Bit 6. (i) Bit 7.

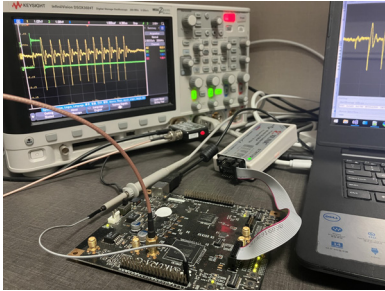


Fig. 10. Complete measurement setup.

to the sensitive variable can be distinguished, verifying the leakage.

There are total 3 000 000 power traces are captured during the executions of RSM on FPGA. Then, we launch the specific t -test for the measurement under BBI model. The intermediate value is set as the bit 1 of the output of the unmasked Sbox. The t value after analyzing 3 000 000 traces is shown in Fig. 11(a). It can be found the result exceeds the threshold, which indicates the practical leakage of RSM's FPGA implementation.

Since it requires a large number of traces to perform a side-channel attack on the practical measurements, a more

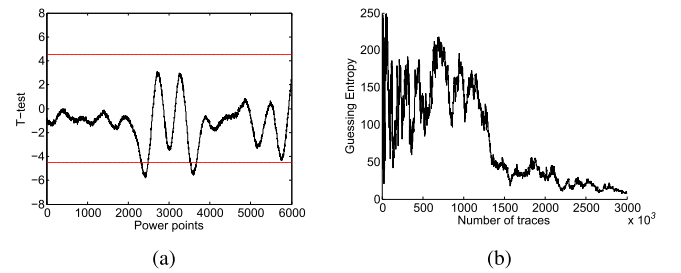


Fig. 11. Practical attack results. (a) t -test under BBI model. (b) Guessing entropy.

flexible metric named guessing entropy [32] is utilized to evaluate the average number of key candidates to test after the attack. The metric was proposed to quantify the effectiveness of side-channel attacks from the perspective of key entropy [33], [34].

We conduct the first-order attack on the set of 3 000 000 power traces. First, we use the statistic alignment as the pretreatment for the power traces. Then, we perform the first-order CPA under the bit 1 of the output of the Sbox. Fig. 11(b) shows the guessing entropies under different number of traces. It can be found that the candidate space of

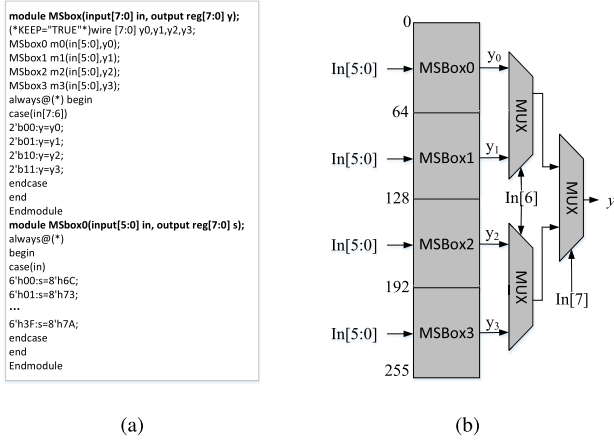


Fig. 12. Modified design to avoid the leakage. (a) Verilog code of the improved implementation. (b) Consistent implementation structure.

the key decreases and tends to the minimum with increasing attack traces. The interesting thing is that the launched attack is the first-order CPA against the unmasked Sbox output. It does not require the combining function in second-order or higher order attacks. The bias among masked circuits makes the leakage of the circuit fail to achieve the bit balance.

V. COUNTERMEASURE

As shown in Proposition 1, the implementation of *BB* secure RSM scheme may not achieve first-order security on the FPGA structure. The imbalanced structure caused by uncontrolled EDA tools makes the theoretical secure scheme fail to achieve first-order security. It makes the circuit's power consumption still related to the sensitive value. In this section, we discuss possible countermeasures for the FPGA implementation of LEMS.

The goal here is to develop the implementation of masked Sbox with a consistent structure to eliminate the leakage. Since the maximum supported LUT is 6-input LUT in modern FPGA, the 256 addresses of the LUT should be divided into four pieces so that each one can be implemented by 6-input LUTs. An exemplary design of the consistent circuit is shown in Fig. 12(a). It can use the low 6 bits as the 6-input LUTs while the high 2 bits as the select signals for the multiplexers. It can avoid risk optimization during the synthesis of uncontrollable EDA tools. The bits used as the inputs for the 6-input LUTs and multiplexers are fixed in this way.

Moreover, the constraint (*KEEP* = "TRUE") is applied to prevent optimizations in circuitual signals that are optimized or absorbed. The masked Sbox implementation generated by design is shown in Fig. 12(b). The consistent structure of different masked Sboxes makes the circuit satisfy *BBI* security.

Proposition 2: In the circuit of *i*th-masked Sbox, the bits X_{LUT}^i are used as the input signals of 6-input LUT and X_{MUX}^i are used as the select signals for the multiplexers. If the implementation structures of 16 masked Sbox are consistent, that is, $\forall i, j \in [0, 15], X_{LUT}^i = X_{LUT}^j, X_{MUX}^i = X_{MUX}^j$, the *BB* secure masking scheme achieves *BBI* security.

Proof: The symbol definitions are consistent with the symbols used in the proof process of Proposition 1. $\forall i, j, X_{LUT}^i = X_{LUT}^j, X_{MUX}^i = X_{MUX}^j$. The *BB* secure masking scheme satisfies

$$\begin{aligned}
 E(t_{LUT}^0 | X[k] = 0) &= E(t_{LUT}^0 | X[k] = 1) \\
 E(t_{LUT}^1 | X[k] = 0) &= E(t_{LUT}^1 | X[k] = 1) \\
 E(t_{LUT}^2 | X[k] = 0) &= E(t_{LUT}^2 | X[k] = 1) \\
 E(t_{LUT}^3 | X[k] = 0) &= E(t_{LUT}^3 | X[k] = 1).
 \end{aligned}$$

The toggles of multiplexers are connected to the output of LUTs and the select signals are the same among different masked Sboxes. Thus,

$$\begin{aligned}
 E(t_{MUX}^0 | X[k] = 0) &= E(t_{MUX}^0 | X[k] = 1) \\
 E(t_{MUX}^1 | X[k] = 0) &= E(t_{MUX}^1 | X[k] = 1) \\
 E(t_{MUX}^2 | X[k] = 0) &= E(t_{MUX}^2 | X[k] = 1).
 \end{aligned}$$

Thus, there has

$$E(H(X_M[k]) | X[k] = 0) = E(H(X_M[k]) | X[k] = 1).$$

The results of applying first-order CPA on the modified implementation during simulation are shown in Fig. 13. We use the same number of traces as the experiment for the original implementation. In contrast to the results of the original flow implementation, the correlation coefficient of the secret key is not visible among the candidates, as shown in Fig. 13(a). We also compute the guessing entropy during the attack. It can be found that the guessing entropy tends to be stable with increasing the number of traces, shown in Fig. 13(b). The secret key cannot be distinguished among the candidate keys efficiently.

We also conduct the practical attacks on the SAKURA-G board. The modified implementation is compiled and downloaded onto the board. The measurement setup is the same with the experiment of the original implementation of RSM. We perform the specific *t*-test on the 3 000 000 power traces. The intermediate value is set as the bit 1 of the output of the unmasked Sbox, which is the same with the above experiment. It can be observed that the result does not exceed the threshold in Fig. 14(a). The first-order CPA is also performed on the trace set. Fig. 14(b) shows the guessing entropies under different number of traces. The candidate space of the key tends to be stable at a high value. The power traces of the modified implementation do not support a successful first-order CPA attack for recovering the secret key.

Circuit optimization by EDA tools would introduce extra threats to the implementation [21], [35]. During the design phase at different levels, the netlist includes not only the variables in the algorithm but also newly generated variables. Moreover, some details may be realized unexpectedly. It is because optimization aims to improve speed or area rather than security. Some options can be selected to set the design goal and strategy. For example, Xilinx supports the setup to

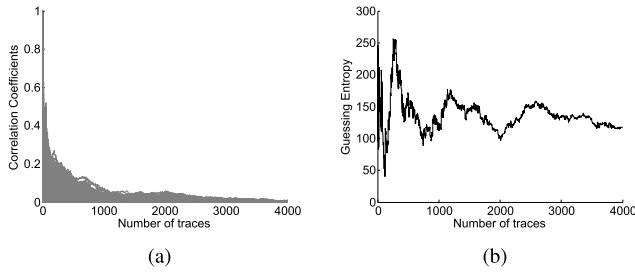


Fig. 13. Attack results on the simulation power traces of the modified implementation. (a) Correlation coefficients. (b) Guessing entropy.

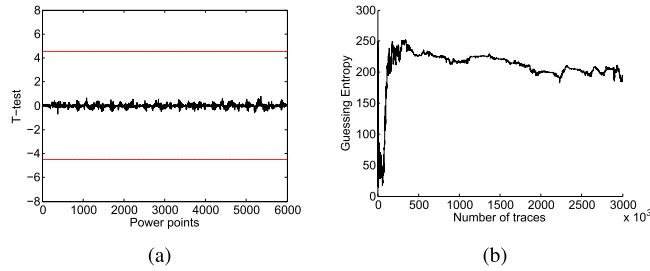


Fig. 14. Practical attack results on the modified implementation. (a) t -test under BBI model. (b) Guessing entropy.

limit the LUT-FF pairs utilization ratio and LUT combining. Generally, the supported strategies are limited and coarse-grained. Knowing the cause of the leakage is a challenge for designers to implement a scheme securely. Thus, it often needs the pre-silicon evaluation to detect the leakage of the cryptographic circuits before manufacture.

During the pre-silicon design phase, there are two ways to leakage detection. The first type is to perform static analysis based on power simulation. It can perform the leakage testing methodology (TVLA) or power analysis attacks (DPA, CPA, et al.) on the simulation results of different levels [36], [37]. Another type is formal verification, which proves the scheme's security by formal method at the algorithm level [38], [39], [40], [41]. Most of them are integrated with the provable security model for the higher order masking scheme. It is not suitable for LEMS. In addition, some schemes verify the masked hardware implementation in the presence of glitches. For example, REBECCA estimates the security of the circuits under the probing model for the worst-case signal timings [41]. It does not cover the structure factor at the implementation level.

The proposed security model can be used as a supplement for the above two types of methods. First, BB security model is the essential condition to resist first-order attacks. It can be used at the algorithm level to detect the flaws of the masking scheme. Then, BBI security model can be integrated into the EDA process to evaluate the security of the implementation of LEMS. After detecting the leakage in the design of different levels, the designer needs to modify their design or implementation to remove the leakage. If the detected leakage is introduced by the EDA optimization, like the inconsistency architecture flaw, the designer could try to remove the leakage by changing the optimization strategy. The designer can add the constraints to disable the optimization

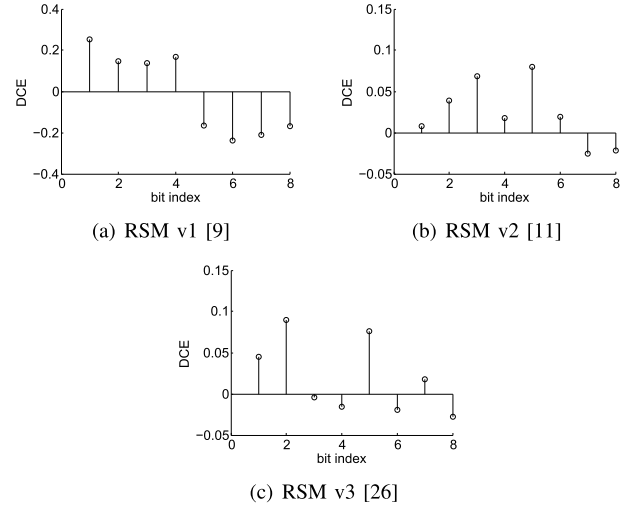


Fig. 15. Analysis of RSM and its variants. (a) RSM v1 [9]. (b) RSM v2 [11]. (c) RSM v3 [26].

and eliminate leakage. For example, the command (e.g., KEEP TRUE) can constrain the crucial signals in the implementation. It can prevent crucial signals from being modified or removed during optimization. It is necessary to feedback to the former security evaluations after modification until there is no leakage in the whole period of design.

VI. DISCUSSION

A. RSM Variants

RSM and its variants are typical of the LEMS implementations. It consumes a small amount of area and performance by limiting the value range of the random mask to 16 fixed elements. In order to guarantee security, it is necessary to achieve BBI security when choosing a random mask in the fixed masking set. Currently, there are four mask sets in RSM and its variants, shown in (4). The latest scheme with MaskSet₄ proposed by Moradi et al. [14] had not been attacked, which is our target during the above analysis. Hereafter, we analyze the RSM scheme using the mask set MaskSet₁ (RSM v1) [9], RSM variant used mask set MaskSet₂ (RSM v2) [11], and RSM variant used mask set MaskSet₃ (RSM v3) [26].

To analyze the leakage of the implementations, we synthesize the implementations and obtain the netlists at the place and route level. The experimental environment is the same as in Section IV. In the netlist, the structures of 16 masked Sboxes are also inconsistent. The structures of the second-byte and 13th-byte masked Sboxes differ from others. The DCE is shown in Fig. 15. The results show that bias exists for each bit under the three schemes.

To prevent inconsistent structures, we replace the masked Sbox with the modified design to avoid leakage. The bias of the mask set cannot be fixed during the implementation level. We evaluate the countermeasure for avoiding the inconsistency leakage. We fix the low six bits as the 6-input LUTs while the high two bits are the select signals for the multiplexors. We also add the constraint (KEEP = "TRUE") to prevent optimizations in circuit signals that are optimized or absorbed. The results are shown in Table I. The obtained netlists show that the structures of different masked Sboxes are consistent.

TABLE I

ANALYSIS ON RSM VARIANTS, FPGA FAMILIES, AND EDA SOFTWARES

RSM variants			
	DCE(max)	BBI	countermeasure
RSM v1	0.2539	✗	✓
RSM v2	0.0801	✗	✓
RSM v3	0.0898	✗	✓
RSM v4	0.0371	✗	✓
FPGA families			
	DCE(max)	BBI	countermeasure
Spartan-6	0.0371	✗	✓
Artix-7	0.0371	✗	✓
Virtex-7	0.0371	✗	✓
Kintex-7	0.0371	✗	✓
EDA software			
	DCE(max)	BBI	countermeasure
ISE 14.7	0.0371	✗	✓
vivado 2012	0.0371	✗	✓
vivado 2024	0	✓	✓
Yosys 0.55 (2025)	0.0430	✗	✓
VTR 9.0 (2025)	0.0273	✗	✓
F4PGA (2023)	0.0215	✗	✓
OpenFPGA (2022)	0.0254	✗	✓
APIO (2024)	0.0293	✗	✓

This work mainly focuses on the security flaw in the FPGA implementation process of LEMS mask scheme, as it uses limited LUT to implement masked Sboxes. For the implementation of FEMS on FPGA, the logic is more complex and the leakage characteristic is different from LEMS, which is one of our future works.

B. FPGA Families

Xilinx has developed more advanced seven series FPGAs to satisfy various application needs. The series includes sub-families Artix-7, Kintex-7, Virtex-7, and so on. Unlike previous families, the seven series FPGAs utilize a unified 28-nm design architecture while ensuring a consistent user experience and eliminating the inconvenience of switching between them.

We also analyze the target implementation of these advanced families. The setting is the same as the experiments on Spartan-6. The results are shown in Table I. It can be observed that the inconsistent structure of the masked Sbox implementation exists in these FPGA series. The input and select signals are the same as the implementation on Spartan-6, leading to the same bias. The improved implementations of the countermeasure are conducted in the FPGAs. The results show that the structures are consistent in the netlist.

When the target FPGA uses an 8-input LUT architecture, we consider two implementation scenarios. First, if the 8-input LUT is composition-based, such as an 8-input fracturable LUT found in the Stratix II family [42], the leakage vulnerability remains. This is because it is constructed using underlying 6-input LUT primitives, which exhibit the same vulnerabilities analyzed in this work. Second, if the 8-input LUT functions as the finest-grained programmable logic unit, there are no structural discrepancies. This is due to the direct match between the LUT's input width and the bit-width of the Sbox inputs, eliminating the mismatch issue.

When designing ASIC implementations, the Sbox LUT is implemented using fine-grained logic primitives (e.g., AND,

OR, NOT gates) from standard cell libraries [43]. This detailed approach results in significant structural variability when compared to FPGA LUT architectures. Implementing Sbox functionality through combinatorial gate networks may reveal multiple potential leakage pathways due to the diverse structures of the circuit. Information leakage similar to vulnerabilities found in FPGA-based systems can still occur, depending on the specific standard cells used and the optimizations applied during synthesis. It needs to access the specific component library files and synthesis tool for further research.

C. EDA Software

We also discuss the results of various EDA software. Vivado was developed for next-generation devices, including UltraScale, UltraScale+, and Zynq-7000 MPSoCs, and integrates a unified design environment leveraging high-level synthesis (HLS), IP-centric workflows, and advanced algorithmic optimizations. Vivados parallelized implementation algorithms reduce compile times by approximately 30% ~ 50% compared to ISE for equivalent designs, while its hierarchical floor-planning tools improve resource utilization by 15% ~ 20%. However, ISE remains prevalent in legacy system maintenance due to its lightweight footprint and compatibility with obsolete toolchains.

We analyze the target implementation on vivado 2012. The compiled netlist shows that the structure of the masked Sbox is inconsistent, and the DCE = 0.0371 indicates the vulnerabilities in the implementation. We also conduct experiments on the latest version, vivado2024. The results show that the structure of the masked Sbox is consistent. The result DCE = 0 indicates that the implementation is BBI secure. We have evaluated the different synthesis strategies on ISE, including “area reduction,” “balanced,” “minimum runtime,” “power optimization,” and “timing performance.” The results are all inconsistent. Vivado 2024 offers more preconfigured synthesis and implementation strategies. The result netlists are consistent under the different synthesis strategies, including “RuntimeOptimized,” “PerformanceOptimized,” “AlternateRoutability,” and “PowerOptimized.”

This inconsistent structure may be related to the synthesis algorithms embedded in EDA software. The synthesis algorithms in commercial software are updated across different versions and not disclosed to the public. To explain the cause of the leakage, we conduct different settings on the widely used open-source software and analyze the relationship between leakage and the synthesis algorithm. Yosys is the first open-source RTL synthesis tool, providing mature support for Verilog and capable of synthesizing complex Verilog designs [44], [45]. It is built as an extensible framework, making it suitable as the foundation for custom synthesis flows. It emphasizes support for coarse-grain logic, which makes it ideal for algorithms like logic mapping to DSP cells in FPGAs or for synthesizing custom coarse-grain reconfigurable hardware.

The EDA software performs numerous operations on MUX structures and has a decisive influence on the selection of control signals in the final netlist [46]. During the synthesis process, the core stage opt performs basic optimizations and

TABLE II
RELATIONSHIP BETWEEN THE LEAKAGE AND SYNTHESIS OPTIMIZATION

parameters	selection signals	DCE(max)
opt -full (default)	MUX1={2,0,1,0,0,0,0,0,1,0,0,0,2,0,0,0}	0.0430
	MUX2={7,2,2,2,2,7,2,2,7,2,2,2,7,2,2,2}	
opt_merge (default)	MUX1={2,0,1,0,0,0,0,0,1,0,0,0,2,0,0,0}	0.0430
	MUX2={7,2,2,2,2,7,2,2,7,2,2,2,7,2,2,2}	
opt_merge -nomux	MUX1={0,0,0,0,0,0,0,0,0,0,0,0,0,2,0,0}	0.0098
	MUX2={2,2,2,2,2,2,2,2,2,2,2,2,2,4,2,2}	
opt_reduce (default)	MUX1={2,0,1,0,0,0,0,0,1,0,0,0,2,0,0,0}	0.0430
	MUX2={7,2,2,2,2,7,2,2,7,2,2,2,7,2,2,2}	
opt_reduce	MUX1={0,0,0,0,0,0,0,0,0,0,0,0,0,2,0,0}	0.0098
	MUX2={2,2,2,2,2,2,2,2,2,2,2,2,2,4,2,2}	
opt_dff (default)	MUX1={2,0,1,0,0,0,0,0,1,0,0,0,2,0,0,0}	0.0430
	MUX2={7,2,2,2,2,7,2,2,7,2,2,2,7,2,2,2}	
opt_dff -keepdc	MUX1={0,0,0,0,0,0,0,0,0,0,0,0,0,2,0,0}	0.0098
	MUX2={2,2,2,2,2,2,2,2,2,2,2,2,2,4,2,2}	

is primarily used to clean up the internal representation of the design between other synthesis passes [44]. According to the document description about the synthesis, the parameters `opt_merge`, `opt_reduce`, and `opt_dff` play an important role in the synthesis `opt` process. To determine the causes of structural inconsistency in the synthesis process, we conduct experiments using various synthesis processes.

The `opt -full` is used as the default configuration in the software. We analyze the synthesized netlist and record the selection signals of the multiplexors MUX1 and MUX2. Then, the maximum of the DCE is calculated to evaluate the inconsistency. We also perform the experiments with different parameters. The results are shown in Table II. The numbers in the selection signals indicate the indices of the input bits of the masked Sboxes.

`opt_merge` identifies cells with identical type and input signals. The `-nomux` option controls whether MUX and PMUX cells are excluded from cell merging. If `-nomux` is not enabled, MUX and PMUX cells are included in the hash-based merging process. This may disrupt the original structural layout and result in changes to the select signals. By enabling `-nomux`, the original connectivity of these multiplexers is preserved, which helps maintain a controllable structure. The results can be found in Table II. The structure in the netlist becomes relatively consistent, except that the selection signals of one byte are inconsistent. The DCE decreases to 0.0098.

`opt_reduce` performs two interrelated optimizations: it consolidates trees of large AND or OR gates with duplicated inputs. It reduces MUX input redundancy by merging identical data inputs and combining their control signals using a logical OR operation. The default setup enables the `-fine` and `-full` options. These optimizations address cases with redundant inputs by reusing and merging duplicated inputs. In contrast, when these options are disabled, the optimizations are applied only if the MUX output is involved in memory write-enable paths. The results are shown in Table II. The DCE decreases to 0.0098.

`opt_dff` converts flip-flops to a more suitable type by merging clock enables and synchronous reset multiplexers, removing unused control inputs, or potentially removing the flip-flop altogether, converting it to a constant driver. The

`-keepdc` option controls whether to preserve undefined values, and this choice significantly affects the optimization's aggressiveness. Enabling `-keepdc` instructs the synthesis to retain behavioral consistency during simulation and prevents the incorrect removal of essential states. The results are shown in Table II. The DCE decreases to 0.0098.

According to the experimental results, it can be observed that the synthesis process leads to the leakage of an inconsistent structure. Yosys employs an iterative optimization approach based on an enhanced version of the Kernighan–Lin heuristic algorithm [47]. This algorithm permutes the mappings from logic nodes to cells and from cells to leaves in the interconnect trees, aiming to reduce the number of connections between switches and thus minimize hardware resource usage. The algorithm is guided by a series of configurable flags that activate specific optimization objectives, and it proceeds through multiple iterations until a stable circuit structure is achieved [48]. The optimization algorithm activates extensive logic rewriting and simplification, particularly targeting MUX. These optimizations may reuse, merge, or decompose MUX inputs, and may also propagate constants or simplify control signals through more compact logic. As a result, the selection signals of MUX can be altered.

We have also conducted the experiments on other EDA softwares. Verilog-to-routing (VTR) efforts to provide an open-source framework for FPGA architecture and CAD research and development [49], [50]. VTR also supports modern commercial architectures, such as Xilinx 7-Series and Altera Stratix 10, enabling a broader range of designs to be used within the VTR flow. F4PGA (FOSS Flows for FPGA) is an open-source toolchain for the development of FPGAs from multiple vendors [51]. OpenFPGA is the first open-source FPGA IP generator with silicon proofs supporting highly customizable FPGA architectures [52], [53]. It provides complete EDA support for customized FPGAs. APIO is a powerful and user-friendly open-source FPGA development toolkit that supports both Verilog and SystemVerilog [54]. It integrates simulation, building, programming, and other FPGA development processes into a unified workflow. These experimental results are shown in Table I. The control signals and structures synthesized by different EDA software vary, resulting in differing leakage.

Generally, the synthesis algorithm of EDA software considers the optimization of performance, area, power, and other dimensions. In contrast, the resistance to side-channel attack has not been considered, especially for the security of low-entropy masking. Therefore, the implementation structure of low-entropy masking on FPGA may be inconsistent or consistent by coincidence. Meanwhile, the EDA software synthesis algorithm is constantly updated and iterated; however, it cannot ensure that future versions are leak-free. Therefore, this work is not limited to one EDA software, which reveals the possibility of leakage of low-entropy masking during FPGA implementation in principle.

VII. CONCLUSION

LEMS is one of the most popular lightweight countermeasures against SCAs. This article has highlighted the leakage of LEMS in FPGA implementation. We extend *BB* secure model to *BB1* secure model at the hardware implementation level. The security models can be used to evaluate the security of LEMS in a sample way before practical leakage assessment. Moreover, it helps to improve the security of implementation during the design phase.

Based on the definitions of the security model and proposition, we theoretically prove LEMS's vulnerability in FPGA implementation. The FPGA design differs from the ASIC counterpart, most notably in using LUTs. It affects the implementation cost by the number of inputs to a Boolean function more seriously than algebraic complexity. We have analyzed the security of FPGA primitives to implement a countermeasure based on RSM. The imbalanced masking caused by uncontrolled EDA tools makes the circuit's power consumption still related to the sensitive variable. The vulnerability of imbalanced architecture makes the RSM implementation fail to resist the first-order attack under *BB1* model. Our proof and experiments suggest that the implementation of the masking scheme needs security evaluation or testing during the design phase. It can be performed to apply for the netlist based on the provided security model, which can also be integrated into the existing EDA tools.

REFERENCES

- [1] Y. Ishai, A. Sahai, and D. Wagner, "Private circuits: Securing hardware against probing attacks," in *Proc. 23rd Annu. Int. Cryptol. Conf. Adv. Cryptol. (CRYPTO)*, Santa Barbara, CA, USA, Springer, Aug. 2003, pp. 463–481.
- [2] M. Rivain and E. Prouff, "Provably secure higher-order masking of AES," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2010, pp. 413–427.
- [3] J.-S. Coron, E. Prouff, M. Rivain, and T. Roche, "Higher-order side channel security and mask refreshing," in *Proc. Int. Workshop Softw. Fast Softw. Encry.*, 2014, pp. 410–424.
- [4] H. Gross, S. Mangard, and T. Korak, "Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order," in *Proc. ACM Workshop Theory Implement. Secur.*, Oct. 2016, p. 3.
- [5] H. Groß, S. Mangard, and T. Korak, "An efficient side-channel protected AES implementation with arbitrary protection order," in *Proc. Top. Cryptol. CT-RSA*, 2017, pp. 95–112.
- [6] R. Ueno, N. Homma, and T. Aoki, "Toward more efficient DPA-resistant AES hardware architecture based on threshold implementation," in *Proc. Int. Workshop Construct. Side-Channel Anal. Secure Design*, 2017, pp. 50–64.
- [7] M. Nassar, Y. Souissi, S. Guilley, and J.-L. Danger, "RSM: A small and fast countermeasure for AES, secure against 1st and 2nd-order zero-offset SCAs," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2012, pp. 1173–1178.
- [8] N. Yamashita, K. Minematsu, T. Okamura, and Y. Tsunoo, "A smaller and faster variant of RSM," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2014, pp. 1–6.
- [9] S. Bhasin, N. Bruneau, J. Danger, S. Guilley, and Z. Najm, "Analysis and improvements of the DPA contest v4 implementation," in *Proc. Int. Conf. Sec. Priv. Appl. Cryptogr. Eng.*, 2014, pp. 201–218.
- [10] X. Ye and T. Eisenbarth, "On the vulnerability of low-entropy masking schemes," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2014, pp. 44–60.
- [11] Z. Liu, N. Gao, C. Tu, Y. Ma, and Z. Liu, "Detecting side channel vulnerabilities in improved rotating s-box masking schemes presenting four non-profiled attacks," in *Proc. Int. Conf. Sel. Areas Cryptogr.*, 2016, pp. 41–57.
- [12] F. Zhang, L. Geng, J. Shen, S. Bhasin, X. Zhao, and S. Guo, "Improved low-entropy masking scheme for LED with mitigation against correlation-enhanced collision attacks," in *Proc. Asian Hardw. Oriented Sec. Trust. Symp.*, Jun. 2017, pp. 49–54.
- [13] J. Ming, Y. Zhou, W. Cheng, H. Li, G. Yang, and Q. Zhang, "Mind the balance: Revealing the vulnerabilities in low-entropy masking schemes," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3694–3708, 2020.
- [14] A. Moradi, S. Guilley, and A. Heuser, "Detecting hidden leakages," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2014, pp. 324–342.
- [15] V. Grosso, F. Standaert, and E. Prouff, "Low-entropy masking schemes, revisited," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2014, pp. 33–43.
- [16] A. Moradi and V. Immler, "Early propagation and imbalanced routing, how to diminish in FPGAs," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2014, pp. 598–615.
- [17] P. Sasdrich, O. Mischke, A. Moradi, and T. Güneysu, "Side-channel protection by randomizing look-up tables on reconfigurable hardware," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2015, pp. 95–107.
- [18] S. Mangard, T. Popp, and B. M. Gammel, "Side-channel leakage of masked CMOS gates," in *Proc. Cryptograph. Track RSA Conf.*, Jun. 2005, pp. 351–365.
- [19] S. Mangard and K. Schramm, "Pinpointing the side-channel leakage of masked AES hardware implementations," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2006, pp. 76–90.
- [20] S. Mangard, N. Pramstaller, and E. Oswald, "Successfully attacking masked AES hardware implementations," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2005, pp. 157–171.
- [21] D. B. Roy, S. Bhasin, S. Guilley, J.-L. Danger, and D. Mukhopadhyay, "From theory to practice of private circuit: A cautionary note," in *Proc. 33rd IEEE Int. Conf. Comput. Design (ICCD)*, Oct. 2015, pp. 296–303.
- [22] Y. Li, Z. Liu, S. Guilley, and M. Tang, "Analysis of multiplicative low-entropy masking schemes against correlation power attack," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 4466–4481, 2021.
- [23] W. Kai and X. Zhiwei, "Synopsys prime power manual release U-2003.06-QA," Synopsys, Sunnyvale, CA, USA, Tech. Rep. P321-E122-04EN, 2003.
- [24] T. Zhang, J. Park, M. Tehranipoor, and F. Farahmandi, "PSC-TG: RTL power side-channel leakage assessment with test pattern generation," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, Dec. 2021, pp. 709–714.
- [25] M. Gruber and G. Sigl, "TOFU—toggle count analysis made simple," *Cryptol. ePrint Arch.*, Feb. 2022, pp. 1–10.
- [26] N. Veshchikov and S. Guilley, "Implementation flaws in the masking scheme of DPA contest v4," *IET Inf. Secur.*, vol. 11, no. 6, pp. 356–362, Nov. 2017.
- [27] S. Bhasin, C. Carlet, and S. Guilley, "Theory of masking with code-words in hardware: Low-weight dth-order correlation-immune Boolean functions," *IACR Cryptol. ePrint Arch.*, vol. 2013, p. 303, Jul. 2013.
- [28] A. Nadja and A. Mohamed, "Efficient implementation of AES S-box in LUT-6 FPGAs," in *Proc. 4th Int. Conf. Electr. Eng. (ICEE)*, Dec. 2015, pp. 1–4.
- [29] H. Guntur, J. Ishii, and A. Satoh, "Side-channel attack user reference architecture board SAKURA-G," in *Proc. IEEE 3rd Global Conf. Consum. Electron. (GCCE)*, Oct. 2014, pp. 271–274.
- [30] G. Goodwill, B. Jun, J. Jaffe, and P. Rohatgi, "A testing methodology for side-channel resistance validation," in *Proc. NIST Noninvasive Attack Testing Workshop*, vol. 7, 2011, pp. 115–136.

- [31] T. Schneider and A. Moradi, "Leakage assessment methodology," *J. Cryptograph. Eng.*, vol. 6, no. 2, pp. 85–99, Jun. 2016.
- [32] F.-X. Standaert, G. Tal Malkin, and M. Yung, "A unified framework for the analysis of side-channel key recovery attacks," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.* Berlin, Germany: Springer, 2009, pp. 443–461.
- [33] J. L. Massey, "Guessing and entropy," in *Proc. IEEE Int. Symp. Inf. Theory*, May 2002, p. 204.
- [34] B. Köpf and D. Basin, "An information-theoretic model for adaptive side-channel attacks," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, Oct. 2007, pp. 286–296.
- [35] J.-L. Danger, S. Guilley, P. Nguyen, R. Nguyen, and Y. Souissi, "Analyzing security breaches of countermeasures throughout the refinement process in hardware design flow," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 1129–1134.
- [36] A. A. Ding, L. Zhang, F. Durvaux, F. Standaert, and Y. Fei, "Towards sound and optimal leakage detection procedure," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2018, pp. 105–122.
- [37] A. Moradi, B. Richter, T. Schneider, and F.-X. Standaert, "Leakage detection with the x2-test," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2018, pp. 209–237, Feb. 2018.
- [38] G. Barthe, S. Belaïd, F. Dupressoir, P.-A. Fouque, B. Grégoire, and P.-Y. Strub, "Verified proofs of higher-order masking," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2015, pp. 457–485.
- [39] S. Faust, V. Grosso, S. M. Del Pozo, C. Paglialonga, and F.-X. Standaert, "Composable masking schemes in the presence of physical defaults & the robust probing model," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2018, pp. 89–120, Aug. 2018.
- [40] G. Barthe, S. Belaïd, P.-A. Fouque, and B. Grégoire, "MaskVerif: A formal tool for analyzing software and hardware masked implementations," *IACR Cryptol. ePrint Arch.*, vol. 2018, p. 562, Jun. 2018.
- [41] R. Bloem, H. Groß, R. Iusupov, B. Könighofer, S. Mangard, and J. Winter, "Formal verification of masked hardware implementations in the presence of glitches," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Techn.*, 2018, pp. 321–353.
- [42] *FPGA Architecture (8-Input LUT)—Legacy White Paper*. Accessed: Jul. 2006. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/771003/fpga-architecture-8-input-lut-legacy-white-paper.html>
- [43] S. Gayathri and T. C. Taranath, "RTL synthesis of case study using design compiler," in *Proc. Int. Conf. Electr., Electron., Commun., Comput., Optim. Techn. (ICEECCOT)*, Dec. 2017, pp. 1–7.
- [44] C. Wolf, J. Glaser, and J. Kepler, "Yosys-a free verilog synthesis suite," in *Proc. 21st Austrian Workshop Microelectron. (Austrochip)*, vol. 97, pp. 1–6, 2013.
- [45] *Yosys Open Synthesis Suite*. Accessed: Jul. 2025. [Online]. Available: <https://github.com/YosysHQ/yosys>
- [46] *YosysHQ Documents: OPT—Perform Simple Optimizations*. Accessed: Jul. 2025. [Online]. Available: <https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/opt.html>
- [47] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *Bell Syst. Tech. J.*, vol. 49, no. 2, pp. 291–307, Feb. 1970.
- [48] J. Glaser and C. Wolf, "Methodology and example-driven interconnect synthesis for designing heterogeneous coarse-grain reconfigurable architectures," in *Models, Methods, and Tools for Complex Chip Design: Selected Contributions From FDL 2012*. Cham, Switzerland: Springer, 2013, pp. 201–221.
- [49] M. A. Elgammal et al., "VTR 9: Open-source CAD for fabric and beyond FPGA architecture exploration," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 18, no. 3, pp. 1–53, Sep. 2025.
- [50] *Verilog to Routing (VTR)*. Accessed: Jan. 2025. [Online]. Available: <https://github.com/verilog-to-routing/vtr-verilog-to-routing>
- [51] *Foss Flows for FPGA (F4PGA) Project*. Accessed: Oct. 2023. [Online]. Available: <https://github.com/chipsalliance/f4pga>
- [52] X. Tang, E. Giacomini, B. Chauviere, A. Alacchi, and P.-E. Gaillardon, "OpenFPGA: An open-source framework for agile prototyping customizable FPGAs," *IEEE Micro*, vol. 40, no. 4, pp. 41–48, Jul. 2020.
- [53] *Getting Started with OpenFPGA*. Accessed: Sep. 2022. [Online]. Available: <https://github.com/Inis-uofu/OpenFPGA>
- [54] *Apio*. Accessed: Jun. 2024. [Online]. Available: <https://github.com/FPGAwards/apio>



Yanbin Li received the Ph.D. degree from the School of Cyber Science and Engineering, Wuhan University, Wuhan, China, in 2018.

He is an Associate Professor with the School of Software, Shandong University, Jinan, China, and the State Key Laboratory of Cryptology, Beijing, China. His research interests include side-channel analysis and cryptanalysis.



Yikang Guo is currently pursuing the master's degree in electronic information with Nanjing Agricultural University, Nanjing, China.

His research interests include side-channel analysis and cryptanalysis.



Fan Zhang (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, Storrs, CT, USA, in 2011.

He is currently a Full Professor with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China, and Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Hangzhou. His research interests include system security, hardware security, network security, cryptography, and computer architecture.